

- AWS Cloud Architect Interview Prep
 - Role: Cloud Architect (Infrastructure & Platform Architecture) — AWS Equivalent
 - Azure → AWS Service Mapping Cheat Sheet
 - Section 1: AWS Architecture & Design
 - Q1. Walk me through how you design a landing zone for a new enterprise AWS environment.
 - Q2. How do you design for multi-region architecture on AWS?
 - Q3. How do you design a Hub-and-Spoke network topology on AWS?
 - Q4. What's your approach to subnet design within a VPC?
 - Section 2: Infrastructure as Code (Terraform / CloudFormation / CDK)
 - Q5. What's your governance model for Terraform at enterprise scale?
 - Q6. CloudFormation vs Terraform vs CDK — when do you use which?
 - Q7. How do you handle secrets in IaC pipelines?
 - Q8. How do you manage Terraform at multi-account scale (20+ accounts)?
 - Section 3: Reliability, High Availability & Disaster Recovery
 - Q9. Walk me through designing a DR strategy for a 3-tier web application on AWS.
 - Q10. How do you design for high availability in EKS?
 - Q11. What's your approach to AWS Backup governance?
 - Section 4: Security & Compliance (NIST on AWS)
 - Q12. How do you implement NIST CSF controls in an AWS environment?
 - Q13. How do you enforce encryption standards across an AWS organization?
 - Q14. How do you approach IAM governance at enterprise scale?
 - Q15. How do you implement network security controls on AWS?
 - Section 5: Platform Architecture & EKS/Containerization
 - Q16. How do you design an EKS platform for multiple application teams?
 - Q17. How do you handle secrets management for containerized workloads?
 - Section 6: Governance, Documentation & Technical Leadership
 - Q18. How do you drive architectural standards adoption across a distributed team?
 - Q19. How do you manage architectural debt in a brownfield AWS environment?
 - Q20. Tell me about a technically complex AWS architecture you designed. What were the trade-offs?

- Section 7: Scenario-Based Questions
 - Q21. A security audit finds that 3 of your production RDS instances have publicly accessible endpoints. How do you respond?
 - Q22. You're asked to reduce AWS costs by 30% without impacting availability. How do you approach this?
- Section 8: Behavioral & Leadership Questions
 - Q23. How do you handle a situation where an application team's design conflicts with your architectural standards?
 - Q24. How do you stay current with AWS as services evolve so rapidly?
- Section 9: Quick-Fire Technical Questions
 - Q25. What's the difference between a Security Group and a NACL?
 - Q26. Explain the difference between S3 Transfer Acceleration, S3 Multipart Upload, and S3 Cross-Region Replication.
 - Q27. What is VPC Peering vs Transit Gateway — when do you use which?
 - Q28. Explain RDS Multi-AZ vs Read Replicas vs Aurora Global Database.
- Preparation Checklist

AWS Cloud Architect Interview Prep

Role: Cloud Architect (Infrastructure & Platform Architecture) — AWS Equivalent

JD was Azure-focused. This guide maps every requirement 1:1 to AWS and provides comprehensive Q&A for each domain area.

Azure → AWS Service Mapping Cheat Sheet

Azure	AWS Equivalent
Azure VNet	VPC
ExpressRoute	AWS Direct Connect

Azure	AWS Equivalent
Azure VPN Gateway	AWS Site-to-Site VPN / Client VPN
Azure Firewall	AWS Network Firewall
Azure Front Door / WAF	AWS CloudFront + AWS WAF
Azure Load Balancer	NLB (Network Load Balancer)
Azure Application Gateway	ALB (Application Load Balancer)
Azure Traffic Manager	Route 53 (routing policies)
Azure Private Endpoint	AWS PrivateLink / VPC Endpoint
Azure DNS	Route 53
Azure Virtual WAN / Hub-Spoke	AWS Transit Gateway
Azure VM / VMSS	EC2 / EC2 Auto Scaling Group
AKS	EKS
Azure Container Apps	ECS Fargate / App Runner
Azure Functions	AWS Lambda
Azure App Service	Elastic Beanstalk / App Runner
Azure Blob Storage	S3
Azure Disk	EBS
Azure Files	EFS / FSx
Azure SQL	RDS / Aurora
Cosmos DB	DynamoDB
Azure Cache (Redis)	ElastiCache (Redis)
Azure Key Vault	AWS Secrets Manager + AWS KMS
Azure AD / Entra	AWS IAM Identity Center (SSO)
ARM Templates	CloudFormation
Bicep	AWS CDK
Terraform	Terraform (same)
Azure Policy	SCP (Service Control Policies) + AWS Config Rules

Azure	AWS Equivalent
Azure Blueprints	AWS Control Tower
Microsoft Defender for Cloud	AWS Security Hub + GuardDuty
Azure Monitor / Log Analytics	CloudWatch + CloudWatch Logs Insights
Azure Sentinel	Amazon Security Lake + Amazon Detective
Macie (same name on AWS)	Amazon Macie
Azure Backup	AWS Backup
Azure Site Recovery	AWS Elastic Disaster Recovery (DRS)
Azure Cost Management	AWS Cost Explorer + Budgets
Management Groups	AWS Organizations

Section 1: AWS Architecture & Design

Q1. Walk me through how you design a landing zone for a new enterprise AWS environment.

Answer:

I follow the AWS Control Tower + AWS Organizations model. The landing zone consists of:

Account Structure (multi-account strategy):

- **Management account** — billing, SCPs, no workloads
- **Log archive account** — centralized CloudTrail, Config, VPC Flow Logs (S3 with object-lock)
- **Audit/Security account** — Security Hub aggregator, GuardDuty delegated admin, Config aggregator
- **Shared Services account** — Transit Gateway hub, DNS (Route 53 Resolver), shared tools (Artifactory, Vault)

- **Workload accounts** — Dev / QA / Staging / Prod, each in separate accounts (not just separate VPCs)

Why separate accounts?

- Blast radius isolation — a misconfigured IAM or runaway cost in Dev can't affect Prod
- Hard security boundary (SCPs enforced at account level, not just IAM)
- Independent billing and cost attribution

Networking:

- Hub-and-spoke using **Transit Gateway** — all workload VPCs attach to a central TGW in the Shared Services account
- Centralized egress via **Network Firewall** in the Shared Services account (inspection VPC)
- **PrivateLink** for all AWS service endpoints to avoid public internet traversal
- **Route 53 Resolver** rules shared via RAM for consistent DNS across accounts

Guardrails:

- Preventive SCPs: deny non-approved regions, deny disabling CloudTrail, deny creating unencrypted S3 buckets
- Detective controls: AWS Config rules for tagging compliance, security group hygiene, public S3 check
- All controls codified in Terraform (Landing Zone Accelerator or custom modules)

Q2. How do you design for multi-region architecture on AWS?

Answer:

First, I classify the workload by RTO/RPO:

Tier	RTO	RPO	Pattern
Tier 1 (critical)	< 15 min	< 5 min	Active-Active
Tier 2 (important)	< 2 hrs	< 1 hr	Pilot Light

Tier	RTO	RPO	Pattern
Tier 3 (standard)	< 8 hrs	< 4 hrs	Warm Standby
Tier 4 (batch/dev)	< 24 hrs	< 24 hrs	Backup & Restore

Active-Active pattern:

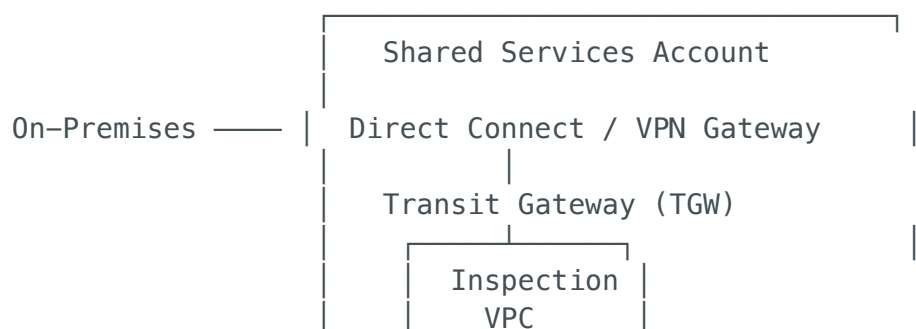
- Route 53 latency-based or geoproximity routing across 2+ regions
- Aurora Global Database (< 1 second replication lag, < 1 minute RTO for regional failover)
- DynamoDB Global Tables (multi-master, eventual consistency)
- S3 Cross-Region Replication with Replication Time Control (RTC — 99.99% of objects in 15 min)
- ElastiCache Global Datastore for Redis
- ALB/NLB in both regions, CloudFront as global entry point

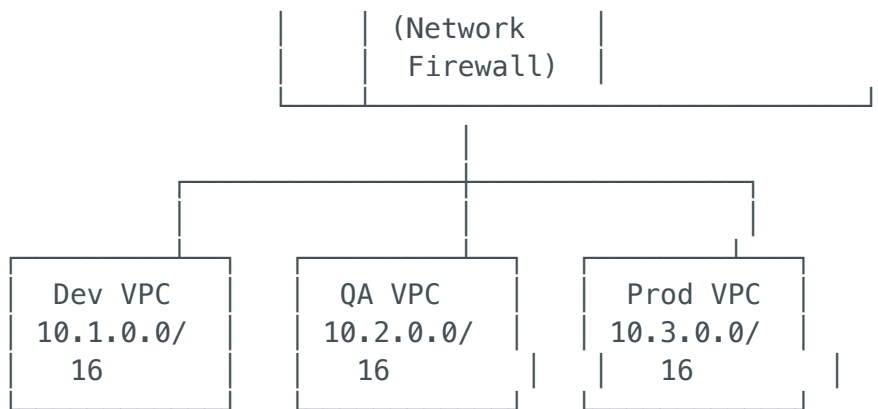
Key design decisions:

- **Data sovereignty** — confirm regulatory requirements allow cross-region data replication
- **Consistency model** — active-active requires accepting eventual consistency for some writes
- **Failover testing** — use AWS FIS (Fault Injection Simulator) to test regional failover quarterly
- **Cost** — running dual-region is ~2x infra cost; justify against business SLA

Q3. How do you design a Hub-and-Spoke network topology on AWS?

Answer:





Key components:

- **Transit Gateway** — the central router; attach all VPCs and on-prem connections
- **TGW Route Tables** — separate route tables for "spoke" VPCs vs "inspection" VPC to control traffic flow
- **Inspection VPC** — egress traffic is routed through AWS Network Firewall before reaching internet via NAT Gateway
- **AWS Resource Access Manager (RAM)** — share TGW and Route 53 Resolver rules across accounts
- **VPC CIDR planning** — non-overlapping /16 per account, documented in an IPAM (AWS VPC IP Address Manager)

Security posture:

- Intra-spoke traffic (Dev → Prod) blocked at TGW route table level (no spoke-to-spoke routes)
- All inter-account flows go through the inspection VPC
- VPC Flow Logs enabled on all VPCs, shipped to centralized Log Archive account S3

Q4. What's your approach to subnet design within a VPC?

Answer:

I use a 3-tier subnet design per AZ:

VPC: 10.0.0.0/16

AZ-1a:

```
Public subnet: 10.0.0.0/24    – ALB, NAT Gateway, Bastion (or SSM)
Private subnet: 10.0.10.0/24 – Application tier (EKS nodes, EC2, Lambda
VPC)
Data subnet: 10.0.20.0/24    – RDS, ElastiCache, OpenSearch

AZ-1b: (mirror)
Public: 10.0.1.0/24
Private: 10.0.11.0/24
Data: 10.0.21.0/24

AZ-1c: (mirror)
Public: 10.0.2.0/24
Private: 10.0.12.0/24
Data: 10.0.22.0/24
```

Rules:

- Public subnets: route 0.0.0.0/0 → IGW; NACLs allow 80/443 inbound
- Private subnets: route 0.0.0.0/0 → NAT Gateway in same AZ (AZ-affinity saves NAT data transfer cost)
- Data subnets: no internet route at all; only inbound from private subnet security groups
- **Dedicated subnets for EKS** — AWS recommends separate subnets for EKS nodes vs. pods (with VPC CNI custom networking)

Section 2: Infrastructure as Code (Terraform / CloudFormation / CDK)

Q5. What's your governance model for Terraform at enterprise scale?

Answer:

Repository structure:

```
infra/
├── modules/           # Reusable, versioned modules (published to private
registry)
│   ├── vpc/
│   ├── eks/
│   └── rds-aurora/
```

```
├── security-baseline/
├── environments/
│   ├── dev/
│   ├── qa/
│   └── prod/
└── live/                                # Terragrunt root configs (DRY wrappers)
```

State management:

- Remote state in S3 with DynamoDB locking (one state file per account/component)
- State bucket has versioning + MFA delete + SSE-KMS encryption
- No state stored locally; enforced via CI policy

Governance controls:

1. **Module versioning** — pin to specific tags (`ref = "v2.1.0"`), never `main`
2. **Mandatory tagging** — `required_tags` variable in every module, validated via `precondition` blocks
3. **Sentinel / OPA policies** — enforce: no public S3, no 0.0.0.0/0 SG ingress, KMS encryption required
4. **CI pipeline gates:**
 - `terraform fmt` check
 - `tflint` + `tfsec` / `checkov` static analysis
 - `terraform plan` output posted to PR as comment
 - `terraform apply` only runs post-merge on protected branches
5. **Drift detection** — scheduled pipeline runs `terraform plan` daily; alert on non-zero diff

My role as architect:

- Review and approve new module PRs
- Set the pattern for module interface design (variable naming conventions, output standards)
- Approve deviation requests from standards

Q6. CloudFormation vs Terraform vs CDK — when do you use which?

Answer:

Tool	When I use it	Strengths	Weaknesses
Terraform	Multi-cloud, brownfield, team already knows it	Provider ecosystem, mature state mgmt, readable HCL	AWS-native resources lag behind CloudFormation
CloudFormation	AWS-only, need native service integration (StackSets, Service Catalog)	First-class AWS citizen, StackSets for multi-account deployment	Verbose YAML/JSON, slower feedback loop
CDK	Teams prefer programming languages, complex conditional logic	Full language power (TypeScript/Python), L3 constructs, great for patterns	Requires build step, generated CFN can be hard to debug

My standard:

- New greenfield AWS-only platform → CDK (TypeScript)
- Multi-cloud or existing Terraform estate → Terraform
- Service Catalog products or StackSets for org-wide policies → CloudFormation
- Never mix Terraform and CDK managing the same resource (state conflict risk)

Q7. How do you handle secrets in IaC pipelines?

Answer:

Hard rule: secrets never in code or state files.

Pattern I use:

1. **AWS Secrets Manager** for runtime secrets (DB passwords, API keys, certificates)
2. **AWS Systems Manager Parameter Store (SecureString)** for config that needs to be referenced at deploy time (non-sensitive but environment-specific values in SSM, secrets in Secrets Manager)

3. In Terraform: use `data "aws_secretsmanager_secret_version"` to reference at plan time, but **never output the value**
4. RDS passwords: use Terraform `random_password` + store to Secrets Manager in same apply; RDS `manage_master_user_password = true` (AWS-managed rotation)
5. **OIDC-based CI authentication** — GitHub Actions → AWS IAM role via OIDC. No static access keys in CI environment variables

What I explicitly forbid:

- AWS access keys in `.tfvars` files
 - Secrets in `terraform.tfstate` outputs
 - Hardcoded passwords in any IaC regardless of environment
-

Q8. How do you manage Terraform at multi-account scale (20+ accounts)?

Answer:

I use **Terragrunt** as a DRY wrapper:

```
# terragrunt.hcl (root)
remote_state {
  backend = "s3"
  config = {
    bucket = "tfstate-${get_aws_account_id()}"
    key    = "${path_relative_to_include()}/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "tfstate-lock"
  }
}
```

- Account-specific values in `account.hcl` (account ID, environment name, CIDR ranges)
- Region-specific values in `region.hcl`
- Component values in local `terragrunt.hcl`
- Terragrunt `run-all` for orchestrating dependency order across components

For org-wide compliance baselines:

- Use CloudFormation StackSets (deploying from management account) for things like: baseline CloudTrail, Config recorder, default VPC deletion, GuardDuty enrollment
 - Terraform handles workload-level infrastructure within each account
-

Section 3: Reliability, High Availability & Disaster Recovery

Q9. Walk me through designing a DR strategy for a 3-tier web application on AWS.

Answer:

Step 1 — Define tiers and objectives with business:

- What is the business cost of 1 hour of downtime? → justifies DR spend
- RTO: how long can the application be unavailable?
- RPO: how much data can we afford to lose?

Step 2 — Choose DR pattern:

Pilot Light (RTO ~30 min, RPO ~5 min):

- DR region has core infrastructure running at minimal capacity (RDS read replica, a minimal ASG with 0 instances)
- Data replication is live (Aurora Global DB replicates continuously)
- On failover: promote Aurora replica, scale up ASG, update Route 53 to DR region
- Cost: ~20-30% of prod cost when idle

Implementation for a typical 3-tier app:

Component	Primary (us-east-1)	DR (us-west-2)	Replication
DNS	Route 53 primary	Route 53 failover record	Health check on ALB
CDN	CloudFront (global — no action needed)	—	—

Component	Primary (us-east-1)	DR (us-west-2)	Replication
Web/App tier	EKS + ALB in 3 AZs	ASG 0 instances (AMIs pre-baked)	Route 53 failover
Database	Aurora MySQL (3 AZs)	Aurora Global DB read replica	< 1 sec lag
Cache	ElastiCache Redis cluster	ElastiCache Global Datastore	Async
File storage	S3 + CRR	S3 (destination)	CRR + RTC
Secrets	Secrets Manager	Secrets Manager (replicated)	AWS-managed
Config	SSM Parameter Store	SSM (replicated parameters)	Lambda replication

Failover runbook (automated with SSM Automation):

1. Detect failure via Route 53 health check or CloudWatch alarm
2. Promote Aurora Global DB in DR region (< 1 minute)
3. Run SSM Automation document: scale ASG, update ALB target group, update Secrets Manager endpoint references
4. Route 53 automatic failover to DR region ALB (TTL 30 seconds)
5. Validate with synthetic canary

Testing:

- **Quarterly:** Full DR failover test (fail traffic to DR region, validate all functionality)
- **Monthly:** Data restore test from AWS Backup
- **Weekly:** Automated AWS FIS experiment (AZ failure simulation)

Q10. How do you design for high availability in EKS?

Answer:

Cluster architecture:

- Control plane: AWS-managed, multi-AZ by default
- Node groups: one Managed Node Group per AZ (allows AZ-specific instance types/capacity)
- Mix On-Demand (base) + Spot (scale): **capacity_type = SPOT** on second node group with **--balance-similar-node-groups** in Cluster Autoscaler

Application-level HA:

```
# Pod Disruption Budget
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api

# Topology Spread Constraints
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: api
```

Key design decisions:

- **minReadySeconds** and **maxUnavailable** in Deployment for zero-downtime rolling updates
- Readiness probes on all pods (prevent routing to non-ready pods during startup)
- Node termination handler (AWS Node Termination Handler) for Spot interruption handling
- Karpenter (preferred over CA) for faster, more cost-efficient node provisioning
- VPC CNI with prefix delegation for IP scalability (avoid IP exhaustion in large clusters)

Q11. What's your approach to AWS Backup governance?

Answer:

Centralized backup policy via AWS Backup + Organizations:

1. Backup plans defined by tier:

- Tier 1: Daily backup retained 35 days + monthly retained 12 months + yearly retained 7 years
- Tier 2: Daily retained 14 days + monthly retained 3 months
- Tier 3: Weekly retained 4 weeks

2. Backup Vault per account with:

- AWS Backup Vault Lock (WORM) for compliance vaults — prevents deletion even by root
- KMS CMK encryption (not AWS-managed key)
- Cross-account/cross-region copy for Tier 1 workloads

3. Tag-based assignment — resource tagged **backup-tier=1** automatically enrolled via Backup plan resource assignment

4. Compliance monitoring:

- AWS Backup Audit Manager reports — automated checks for backup compliance
- Config rule: **backup-plan-exists** for all RDS instances
- Weekly restore test for production databases (automated via Lambda)

5. Recovery validation — monthly automated test: restore RDS snapshot to isolated VPC, run smoke test, delete

Section 4: Security & Compliance (NIST on AWS)

Q12. How do you implement NIST CSF controls in an AWS environment?

Answer:

NIST CSF maps cleanly to AWS services across 5 functions:

IDENTIFY

- AWS Config: inventory of all resources (enabled org-wide via delegated admin)
- AWS Systems Manager Inventory: software inventory on EC2
- Tag policy enforcement via Organizations Tag Policies
- AWS Security Hub: security posture score (NIST SP 800-53 standard built-in)

PROTECT

- IAM: least privilege, permission boundaries, no long-lived access keys
- SCPs: preventive guardrails (deny non-compliant regions, deny disabling security services)
- AWS KMS: encryption at rest for all data stores (CMK with annual rotation)
- VPC: private subnets, Security Groups (deny-by-default), NACLs
- AWS WAF: on CloudFront and ALBs (OWASP Top 10 managed rule groups)
- Secrets Manager: credential rotation
- AWS Certificate Manager: TLS everywhere

DETECT

- Amazon GuardDuty: threat detection (ML-based; enable across all accounts via delegated admin)
- AWS Security Hub: aggregated findings, compliance scoring
- Amazon Macie: PII/sensitive data discovery in S3
- CloudTrail: API audit log (all regions, all services, encrypted, immutable)
- VPC Flow Logs → S3 → Athena for network forensics
- Config Rules: continuous compliance evaluation

RESPOND

- AWS Security Hub → EventBridge → Lambda/SNS for automated response
- GuardDuty finding → Lambda → isolate EC2 (remove from ASG, attach restrictive SG)
- Inspector finding → Jira ticket via Lambda integration
- Runbooks in SSM Automation for common incident types

RECOVER

- AWS Backup with cross-account/cross-region copies

- Aurora Global Database failover
 - Documented and tested DR runbooks in SSM Automation
-

Q13. How do you enforce encryption standards across an AWS organization?

Answer:

Preventive (SCPs):

```
{
  "Effect": "Deny",
  "Action": [
    "s3:PutObject"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": ["aws:kms", "AES256"]
    }
  }
}
```

Additional SCPs: deny unencrypted EBS volume creation, deny RDS without StorageEncrypted=true, deny creating KMS keys with key rotation disabled.

Detective (Config Rules):

- `s3-bucket-server-side-encryption-enabled`
- `rds-storage-encrypted`
- `ebs-encrypted-volumes`
- `encrypted-volumes`
- `cmk-backing-key-rotation-enabled`

KMS architecture:

- One KMS CMK per service per account (separate keys for S3, RDS, EBS, Secrets Manager)
- Key policies: least privilege; no `kms:*` to root unless break-glass
- Cross-account key sharing via key policy (not IAM policy)
- Annual automatic key rotation enabled

- KMS CloudTrail integration — every decrypt logged
-

Q14. How do you approach IAM governance at enterprise scale?

Answer:

Principles:

1. **No IAM users** — all human access via IAM Identity Center (SSO) with permission sets
2. **No long-lived access keys** — EC2/Lambda/ECS use instance roles/execution roles; CI uses OIDC
3. **Permission Boundaries** — prevent privilege escalation; all developer-created roles must have a boundary policy attached
4. **SCPs** — org-level guardrails (IAM cannot override SCPs)

Permission set model in IAM Identity Center:

- **PlatformEngineer** — wide infra permissions in Dev; limited in Prod
- **DeveloperReadOnly** — read-only access to their account's application resources
- **BreakGlass** — emergency admin access; requires MFA + justification; CloudWatch alarm on use
- **SecurityAudit** — read-only across all accounts (mapped to AWS-managed SecurityAudit policy)

Access reviews:

- Quarterly: IAM Access Analyzer findings review
- Monthly: identify and remediate IAM Access Analyzer unused access (roles/users not used in 90 days)
- Weekly: Config rule **iam-root-access-key-check** and **root-account-mfa-enabled**

Automated remediation:

- Config Rule detects non-compliant IAM resource → SSM Automation remediation document → disable/flag
- GuardDuty **UnauthorizedAccess:IAMUser** → Lambda isolates credentials

Q15. How do you implement network security controls on AWS?

Answer:

Defense in depth (4 layers):

1. Edge layer (CloudFront + WAF)

- AWS WAF with: AWS Managed Rules (Core rule set, Known bad inputs, IP reputation list)
- Rate-based rules (throttle by IP)
- Geo-blocking for regions with no business need
- CloudFront with HTTPS-only, TLS 1.2+ minimum

2. Perimeter layer (AWS Network Firewall)

- Centralized in Shared Services / Inspection VPC
- Stateful rules: block known malicious IPs (AWS Managed threat intel)
- Domain-based filtering for egress (allow-list of approved domains for outbound)
- Suricata-compatible IDS/IPS rules

3. Subnet layer (Security Groups + NACLs)

- Security Groups: default deny, only required ports/protocols from specific SG sources (not CIDRs)
- NACLs: additional layer for explicit deny lists
- No **0.0.0.0/0** inbound on any non-public-facing SG
- Regular Config rule evaluation: **restricted-ssh**, **vpc-sg-open-only-to-authorized-ports**

4. Instance/pod layer (host-based)

- IMDSv2 enforced on all EC2 (prevent SSRF-based metadata theft)
 - EKS: network policies (Calico or VPC CNI network policy) to restrict pod-to-pod traffic
 - SSM Session Manager for all EC2 access — no SSH/22 open, no bastion hosts
-

Section 5: Platform Architecture & EKS/Containerization

Q16. How do you design an EKS platform for multiple application teams?

Answer:

Multi-tenancy model:

- One EKS cluster per environment tier (Dev/QA/Prod) — not one cluster per team
- Namespaces per application team
- RBAC: teams have **edit** role in their namespace only; **view** in others

Shared cluster platform services (installed once):

Platform Layer:

— AWS Load Balancer Controller	(ALB/NLB provisioning)
— External DNS	(Route 53 record management)
— Cert Manager	(ACM/Let's Encrypt TLS)
— Cluster Autoscaler / Karpenter	(node scaling)
— AWS EBS CSI Driver	(persistent volumes)
— AWS EFS CSI Driver	(shared file storage)
— External Secrets Operator	(sync from Secrets Manager → K8s secrets)
— Kyverno / OPA Gatekeeper	(policy enforcement)
— Prometheus + Grafana	(metrics)
— Fluent Bit → CloudWatch	(log aggregation)
— AWS Distro for OpenTelemetry	(tracing → X-Ray)

Policy enforcement via Kyverno:

- Require resource requests/limits on all pods
- Require specific labels (team, app, env) on all resources
- Deny privileged containers, hostPath volumes, hostNetwork
- Require non-root user in container spec

GitOps delivery:

- ArgoCD with ApplicationSets — one AppSet per team, targeting team namespace

- ArgoCD projects: restrict what repositories and namespaces each team can deploy to
 - ArgoCD RBAC: team leads can sync/create, developers can view
-

Q17. How do you handle secrets management for containerized workloads?

Answer:

AWS-native approach:

1. Secrets stored in **AWS Secrets Manager** (not Kubernetes Secrets — those are only base64 encoded)
2. **External Secrets Operator (ESO)** — syncs Secrets Manager secrets into Kubernetes Secrets on a schedule
3. EKS pods use **IAM Roles for Service Accounts (IRSA)** — pod gets a temporary IAM role credential via OIDC
4. IRSA role policy: least privilege access only to the specific Secrets Manager secrets that pod needs

```
# ExternalSecret object
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials
  data:
    - secretKey: password
      remoteRef:
        key: prod/myapp/db
        property: password
```

Why not mount Secrets Manager directly via CSI?

- ASCP (AWS Secrets Manager CSI Driver) mounts as file — works but limits rotationresponsiveness

- ESO gives more control over sync intervals and works with GitOps patterns better
-

Section 6: Governance, Documentation & Technical Leadership

Q18. How do you drive architectural standards adoption across a distributed team?

Answer:

The problem: Architects define standards, engineers ignore or circumvent them due to friction.

My approach:

1. Make the right thing the easy thing:

- Publish internal Terraform module registry — engineers get pre-approved, pre-tested modules. Building from scratch is harder than using the module.
- Golden AMIs baked with all security agents pre-installed — engineers can't forget to install them
- Service Catalog products for common patterns (VPC, RDS, EKS cluster) — self-service within guardrails

2. Automate enforcement, not manual reviews:

- Shift-left: tfsec/checkov in IDE extensions catches issues before PR
- OPA/Sentinel in CI blocks non-compliant IaC from ever being applied
- Config Rules detect drift and create Jira tickets automatically

3. Architecture decision records (ADRs):

- All significant decisions documented as ADRs in Git
- Format: Context, Options considered, Decision, Consequences
- Engineers can propose ADRs — democratizes architectural input

4. **Weekly office hours** — 30-minute open session for engineers to bring design questions before they build the wrong thing

5. **Metrics for adoption:**

- % of deployments using approved modules (tracked via Terraform registry download counts)
 - Config Rule compliance score by account (Security Hub aggregated view)
 - Time to resolve architectural drift (detected → remediated)
-

Q19. How do you manage architectural debt in a brownfield AWS environment?

Answer:

Discovery phase:

- AWS Config + Config Rules: enumerate all resources and flag non-compliant ones
- Trusted Advisor: identify underutilized resources, security gaps, service limits
- AWS Compute Optimizer: rightsizing recommendations (EC2, Lambda, ECS, EBS)
- Manual architecture review of each application team's AWS account

Prioritization matrix:

Issue	Security Risk	Operational Risk	Effort	Priority
Unencrypted RDS instances	High	Low	Medium	P1
SSH open to 0.0.0.0/0	High	Low	Low	P1
Manually provisioned resources (no IaC)	Low	High	High	P2
Oversized EC2 instances	Low	Low	Low	P3

Remediation approach:

- P1 (security): remediate in 30 days; use SSM Automation runbooks for automated fix where possible

- P2 (operational): import existing resources into Terraform state (**terraform import**) — don't rebuild
- P3 (optimization): bundle with next planned change to that resource

Prevent recurrence:

- SCPs prevent the highest-risk patterns from being recreated
 - Config Rules with auto-remediation for common issues
 - Architecture review required for new workloads (lightweight — 1-page design doc + 30-min review)
-

Q20. Tell me about a technically complex AWS architecture you designed. What were the trade-offs?

Answer (framework for structuring your own story):

Structure your answer using the STAR method + trade-off articulation:

Situation: "We had a financial services application with strict compliance (SOC 2, PCI DSS Level 1) running on-premises, needed to migrate to AWS with zero tolerance for data breach and RTO < 15 minutes."

Task: "I was responsible for designing the target AWS architecture, including network, security controls, DR, and the migration approach."

Architecture I designed:

- Dedicated VPC with no internet gateway — all access via AWS Direct Connect (redundant, two providers)
- VPC endpoints for all AWS services (no traffic ever traverses the internet)
- Aurora PostgreSQL Multi-AZ with read replicas; Aurora Global Database to DR region
- EKS with node groups in private subnets; no public endpoint for API server
- AWS PrivateLink to expose application APIs to partner organizations
- AWS Macie scanning all S3 buckets containing cardholder data
- Separate AWS account for the PCI workload (scope isolation)

Trade-offs I made:

1. **Direct Connect only (no VPN backup)** — DX is not instant to provision; we accepted the risk and mitigated by ordering two DX connections from different providers at different PoPs
2. **No public EKS endpoint** — operations complexity increased (engineers need to be on VPN to run kubectl); justified by security requirement
3. **Cross-region Aurora read replica** — adds ~\$800/month but delivers RPO < 5 minutes vs RPO of 24 hours with snapshots only

Result: "Passed PCI DSS Level 1 audit first attempt. DR test achieved RTO of 11 minutes against the 15-minute SLA."

Section 7: Scenario-Based Questions

Q21. A security audit finds that 3 of your production RDS instances have publicly accessible endpoints. How do you respond?

Answer:

Immediate (0-4 hours):

1. Verify via Console/AWS Config — confirm **PubliclyAccessible = true** and check Security Groups: is port 5432/3306 actually open to 0.0.0.0/0?
2. If SG allows public access: immediately update SG to remove public rule. This is the blast radius containment.
3. Check CloudTrail for any external connections to those RDS endpoints in last 30 days
4. Notify CISO and open incident ticket — even if no breach confirmed, this is a material finding

Short-term (1-2 weeks):

1. Remediate: modify RDS instance to **PubliclyAccessible = false**, update Terraform state
2. Root cause: why was this deployed this way? Was it a misconfigured Terraform module, a manual change, or a gap in standards?

3. Immediate: add SCP to deny `rds:CreateDBInstance` and `rds:ModifyDBInstance` when `PubliclyAccessible = true`
4. Add Config Rule: `rds-instance-public-access-check` with auto-remediation via Lambda

Long-term (architectural fix):

1. All RDS instances accessed via Application layer only (no direct DB access from developer machines)
 2. DBA access via SSM Session Manager port-forwarding or RDS Proxy — no direct DB exposure
 3. Terraform module for RDS: hardcode `publicly_accessible = false`, no variable to override
-

Q22. You're asked to reduce AWS costs by 30% without impacting availability. How do you approach this?

Answer:

Phase 1 — Discover (Week 1-2):

- AWS Cost Explorer: identify top 5 cost drivers (usually EC2, RDS, Data Transfer, NAT Gateway, S3)
- Compute Optimizer: rightsizing recommendations for EC2, Lambda, ECS, EBS
- Trusted Advisor: idle/underutilized resources
- Cost & Usage Report in Athena: identify unused resources (EC2 stopped but EBS still attached, unattached EIPs, old snapshots)

Typical findings and savings:

Category	Finding	Action	Typical Saving
EC2	Over-provisioned instance sizes	Rightsize (Compute Optimizer)	15-30%
EC2	Dev/QA always-on	Scheduled stop (Lambda + EventBridge)	60-70% of dev cost

Category	Finding	Action	Typical Saving
RDS	Dev/QA full production size	Rightsize + use Aurora Serverless v2 for dev	40-60%
Savings Plans	No commitment	Purchase 1-year Compute Savings Plan	30-40% of EC2+Fargate
NAT Gateway	High data processing	VPC Endpoints for S3/DynamoDB (free)	Eliminate NAT transfer cost
Data Transfer	Cross-AZ transfer	AZ-affinity for cache, same-AZ reads	30-50% of transfer cost
S3	Objects in wrong tier	S3 Intelligent-Tiering or lifecycle policies	40-70% on cold data
EBS	gp2 volumes	Migrate to gp3 (20% cheaper, better perf)	20% on EBS
Snapshots	No retention policy	AWS Backup lifecycle rules	Variable

Phase 2 — Quick wins (Month 1):

- Apply Compute Savings Plans (1-year, no upfront) — immediate 30-40% on compute
- Scheduled shutdown for dev/QA environments (9 PM - 7 AM + weekends)
- Migrate gp2 → gp3 (no downtime, CLI command per volume)
- Enable S3 Intelligent-Tiering on buckets > 128KB average object size

Phase 3 — Structural (Month 2-3):

- Rightsize RDS instances; dev uses Aurora Serverless v2 (scales to zero)
- Spot Instances for stateless workloads (EKS worker nodes, Batch, CI workers) — 70% discount
- VPC Endpoints for all S3/DynamoDB traffic (eliminates NAT Gateway processing cost for AWS traffic)

Governance:

- Set AWS Budgets with alerts at 80%, 90%, 100% of target
- Cost allocation tags enforced; untagged resources flagged weekly
- Monthly cost review with application teams (show their account's cost trend)

Section 8: Behavioral & Leadership Questions

Q23. How do you handle a situation where an application team's design conflicts with your architectural standards?

Answer:

Acknowledge the tension first — architecture standards exist to serve the business, not the other way around. My approach:

1. **Understand the constraint first:** Why are they proposing the non-standard approach? Time pressure? A technical limitation with the standard? A genuine edge case?
2. **Present the risk, not the rule:** Instead of "that violates standard X", say "that approach creates Y risk because Z — here's what happened to Company A". Risk language resonates with stakeholders more than policy language.
3. **Find a path:** Can the standard be met with a small modification? Can we issue a time-limited exception with a remediation commitment?
4. **Escalate when necessary:** If the risk is security or compliance related and the team won't budge, escalate to CISO/CTO with clear risk statement. This is not about winning — it's about ensuring decision-makers are informed.
5. **Document the decision either way:** If an exception is granted, document it as an ADR with the risk acknowledgement and remediation timeline. Exceptions shouldn't be invisible.

Q24. How do you stay current with AWS as services evolve so rapidly?

Answer:

My practical approach:

- **AWS re:Invent / re:Inforce** — watch keynotes and 300/400-level sessions for services relevant to my domain
- **AWS What's New RSS feed** filtered to my top 10 services — daily 5-minute scan
- **AWS blog** (particularly the Architecture and Security blogs) — weekly
- **Hands-on:** I maintain a personal sandbox AWS account; any interesting new service gets a proof-of-concept before I recommend it in production
- **Community:** AWS Community Builders program, relevant subreddits (r/aws), Twitter/X architect community
- **Certifications:** I hold AWS Solutions Architect Professional and Security Specialty — re-certification keeps knowledge current

How I translate this to the team:

- Monthly 30-minute "AWS News That Matters" team session — filter the noise, present only what's relevant to our platform
- If a new service solves a current pain point, I write an internal tech spike proposal

Section 9: Quick-Fire Technical Questions

Q25. What's the difference between a Security Group and a NACL?

	Security Group	NACL
State	Stateful (return traffic automatically allowed)	Stateless (must explicitly allow both directions)
Level	Instance/ENI level	Subnet level
Rules	Allow rules only	Allow and Deny rules
Evaluation	All rules evaluated	Rules evaluated in order (lowest number first)
Use case	Primary workload access control	Additional perimeter, explicit deny lists

Q26. Explain the difference between S3 Transfer Acceleration, S3 Multipart Upload, and S3 Cross-Region Replication.

- **Transfer Acceleration:** uses CloudFront edge locations to accelerate uploads from distant clients to S3. Client uploads to nearest edge, travels AWS backbone to S3. Good for global users uploading large files.
- **Multipart Upload:** splits large objects (> 100MB) into parts uploaded in parallel. Improves throughput and allows resuming failed uploads. Required for objects > 5GB.
- **Cross-Region Replication (CRR):** asynchronously replicates objects to a bucket in a different region after they are written. Used for DR, latency reduction, compliance (data residency). CRR + Replication Time Control (RTC) guarantees 99.99% of objects replicated within 15 minutes.

Q27. What is VPC Peering vs Transit Gateway — when do you use which?

	VPC Peering	Transit Gateway
Topology	Point-to-point	Hub-and-spoke (many-to-many)
Routing	Non-transitive (A-B, B-C ≠ A-C)	Transitive (A→TGW→C)
Scalability	$N*(N-1)/2$ connections for N VPCs	One TGW, unlimited attachments
Cost	Free (data transfer cost only)	$0.05/hrperattachment + 0.02/GB$ processed
Use case	2-3 VPCs, same account, simple	5+ VPCs, multi-account, hybrid connectivity
Cross-region	Yes (with limits)	Yes (TGW peering between regions)

My guidance: Use VPC Peering only for < 3 VPCs in simple scenarios. Any enterprise or growing environment → Transit Gateway from the start. Retrofitting TGW after building a

Q28. Explain RDS Multi-AZ vs Read Replicas vs Aurora Global Database.

	RDS Multi-AZ	Read Replicas	Aurora Global DB
Purpose	HA (failover)	Read scaling + DR	Multi-region HA + read scaling
Replication	Synchronous	Asynchronous	Asynchronous (< 1 sec)
Readable	No (standby not readable)	Yes	Yes (secondary regions)
Failover RTO	~1-2 min (automatic)	Manual (promote)	< 1 min (managed failover)
RPO	0 (synchronous)	Seconds-minutes of lag	< 1 second
Scope	Single region, multi-AZ	Same or different region	Multi-region
Use for	Standard HA	Read scaling, DR option	Global/critical apps with multi-region DR

Preparation Checklist

- ☐ Practice the landing zone design (Q1) out loud — 5-minute verbal walkthrough
 - ☐ Prepare a real project story for Q20 (complex architecture + trade-offs)
 - ☐ Know the NIST CSF → AWS service mapping (Q12) cold
 - ☐ Be ready to draw the Hub-and-Spoke TGW topology on a whiteboard/miro
 - ☐ Rehearse the brownfield DR scenario (Q21 — security incident) — shows maturity
 - ☐ Have cost optimization numbers ready (Q22) — interviewers love specifics
 - ☐ Review the Azure → AWS mapping table — they may test this explicitly since the JD was Azure
-

